A Technical Audit of a Wolfram Language Radical-Denesting Prototype

`Strad`, `DenestRadicals3`, and `denest11`

Algorithm reconstruction, mathematical justification,
branch counterexamples, implementation defects, and a repair strategy

September 5, 2026

Material reviewed. The single source file supplied with the request, reproduced without modification as `source.wl` in the accompanying archive. The file has 585 physical lines. All source-line references in this article refer to that copy, not to a reformatted version.

Evidence. The review combines complete static inspection of the supplied source, mathematical derivations, current official Wolfram Language documentation, and 26 successful independent checks in Python/SymPy. The supplied program was *not* executed in a Wolfram kernel in this session. Wolfram regression tests are included for reproducing the predicted failure paths; they are not presented as an executed test report.

Main conclusion. The minimal-polynomial/GCD idea is mathematically useful, but the present implementation is not a trustworthy general-purpose simplifier. It can validate the wrong branch of the original number, its factorization layer uses branch-unsafe transformations, and its search machinery contains independent ordering and indexing defects. These are repairable without discarding the central algebraic idea.

Source SHA-256:

`693a2825f1e2b8912e55371b90dc8ed115bc`
`33ee707dd3a4521eb22bcd2ccc93`

Abstract

The supplied Wolfram Language program attempts to denest exact algebraic radical expressions by combining syntactic normalization with a multiplier search. For each multiplier it computes a minimal polynomial, intersects that polynomial with a binomial over an algebraic coefficient field, solves a lower-degree common factor, and selects a candidate using a zero test. This article reconstructs that process, proves the mathematical statements that actually justify it, and separates them from the heuristic decisions that drive the search.

The most serious defect is a change of target: after replacing an original number α by $\rho = \alpha^q$, the implementation validates candidates against the principal value $\rho^{1/q}$ instead of against α. Exact counterexamples show how this reverses a sign even when every minimal-polynomial and GCD computation is correct. Other findings include unrestricted radical factorization, a comparator that ignores its second argument, a stale-length slice after prime pruning, unchecked empty solution lists, missing multiplier validation, and an ineffective candidate cap. The article also identifies sound components, including denominator rationalization by a minimal polynomial, and gives a conservative candidate kernel, regression cases, and a prioritized redesign that makes correctness independent of search quality.

Contents

1 Scope, evidence, and the intended contract

1.1 What the attachment establishes

The attachment contains definitions, comments, and diagnostic print statements, but no usage messages, version declaration, sample invocation, test suite, specification of supported inputs, or stated mathematical completeness theorem. Accordingly, the review does not infer an author, a release history, or an intended domain beyond what the code itself supports. In particular, the suffixes `3` and `11` are names, not evidence of documented algorithm versions.

The public-looking entry point is

```
Strad[expr, alllevels, func]
```

with the actual optional arguments defined by

```
DenestRadicals3[expr_, alllevels_: False, func_: denest11]
```

The wrapper first performs custom factorization, then locates rational powers of radicands for which `AlgebraicQ` succeeds. It can therefore occur in a symbolic host expression, although the algebraic core is naturally suited to explicit, finite, exact algebraic numbers. Those two domains are not equivalent, and the code does not formally distinguish them.

A reasonable correctness contract would be:

> For every supported input, return an expression with exactly the same value and principal-branch interpretation. Replace it by a different representation only when a declared simplification objective improves. If a bounded search does not find such a representation, return the unchanged input with an explicit status.

That is a proposed contract, not a claim that the attachment already implements it.

1.2 Four levels of evidence

Throughout the article, a *source finding* is a consequence of the definitions and control flow. A *mathematical finding* is justified by a derivation given here. An *independent check* is an executed calculation in the accompanying Python script. A *Wolfram regression* is a supplied test whose actual kernel result remains to be collected.

This distinction matters particularly for end-to-end examples. Exact identities and polynomial GCDs can establish that an implemented branch choice is wrong without establishing the precise printed transcript of a complete Wolfram evaluation. Automatic simplification, factor ordering, and diagnostics can vary with kernel version and expression representation. None of the predicted transcripts in this article is described as an observed Wolfram run.

Official documentation is used to resolve language semantics, not to silently replace the source algorithm. The core mathematical analysis and counterexamples are derived from the supplied definitions. The companion files preserve this separation: `verification_results.json` records only independent checks; `regression_tests.wlt` contains unexecuted Wolfram assertions.

1.3 Findings at a glance

ID	Priority	Finding	Source lines
F1	Critical	Candidates are certified against a reconstructed principal root rather than the original target.	90–94, 122–127
F2	Critical	The custom factorizer distributes and combines powers without the branch conditions needed for general inputs.	527–543, 581–583
F3	High	The prime comparator uses only its first operand, comparing a prime with its own exponent.	177–181
F4	High	Prime pruning computes a slice limit using the pre-pruning list length.	184–194
F5	High	Empty candidate lists are indexed at position 1; zero and other invalid multipliers are not rejected.	107–134, 214–215
F6	High	The default zero test is not an explicit exact-algebraic certificate; there is no final certificate against the original input.	127, 314, 440
F7	High	The nominal multiplier cap is not a cap; divisor and subset generation can allocate enormous lists.	162–199, 402–420
F8	Medium	A lower-degree GCD is called a denesting without measuring the returned representation.	120–134, 309–324
F9	Medium	Nested-power detection and the later rational-power algorithm have mismatched input contracts.	84–108
F10	Medium	Loading the source changes the global protection status of `Print`.	1–3
F11	Medium	Search heuristics discard possibilities and expose no completeness or exhaustion certificate.	169–208, 224–248, 337–425
F12	Low	Several controls are unused or misleading, notably `extrapower` and `maxprimes`.	95–102, 162–165

The priorities distinguish wrong answers from crashes, resource exhaustion, and incomplete simplification. A heuristic denester is allowed to miss a simplification; it is not allowed to replace a number by a different branch and call that success.

2 Architecture and data flow

2.1 The functions and their responsibilities

Function	Responsibility	Lines
`Strad`	Suppress diagnostic `Print` calls while invoking the wrapper.	1–3
`DenestRadicals3`	Normalize an expression, mark radical subproblems, group some products, invoke a solver, and reassemble.	4–76
`denest11`	Choose a reduction exponent and manage a search over multipliers and algebraic degrees.	78–441
`trymultiplier`	Compute a minimal polynomial and a polynomial GCD, then generate and filter candidates. Local to `denest11`.	107–155
`newmultipliers`	Generate new trial multipliers from a polynomial discriminant or from two trial records. Local to `denest11`.	159–211
`RationalizeDenominator`	Express the inverse of an algebraic denominator using its minimal polynomial.	443–451
`AlgebraicQ`	Ask whether an expression belongs to the algebraic numbers.	453–454
`nestdepths`	Annotate an expression tree with a predicate-dependent nesting height.	456–466
`nestdepthsort`	Flatten and group that annotation by height.	468–480
`maxnestdepth`	Extract the portions retained at the maximum selected nesting height.	482–498
`ComplexitySort`	Order expressions using leaf count, then absolute value.	500–504

Function	Responsibility	Lines
`Factorc`	Decode the custom nested factor representation into an expression.	506–516
`fullfactor`	Construct and repeatedly refine that representation.	518–551
`combine`	Extract common bases and exponents from factored summands.	553–584

The local `SumQ` predicate in the wrapper, lines 13–15, is a normalization helper rather than another denesting algorithm.

2.2 The top-level pipeline

The wrapper's first operation is `Factorc[expr]`, before the algebraicity test on individual radicands. It chooses `ReplaceAll` unless `alllevels` is literally `True`; in that case it chooses `ReplaceRepeated`. A matching rational power is temporarily represented as

$$f[\text{normalized radicand}, \text{rational exponent}].$$

The symbol `f` is a dynamically localized marker, not the mathematical function being computed.

Normalization extracts a content factor, factors parts of that content, merges factors recognized as sums or non-purely-imaginary complex atoms, and flips signs of factors with numerically negative real part. It then multiplies the factors back together, applies denominator rationalization, and simplifies. The sign flips are accompanied by a compensating sign accumulator `j`; that specific operation is not, by itself, a value-changing rewrite.

Next, some products of two marked radicals are bundled into a single one-argument marker. The exact rule constructs

$$f[\mathrm{rad}_1^{p_1}\mathrm{rad}_2^{p_2}],$$

not $f[(\mathrm{rad}_1\mathrm{rad}_2)^p]$. This is an important distinction: the product-bundling rule itself does not assert an invalid product-of-roots identity. The branch problem appears later when the bundled product is treated as the principal root of its own power.

The remaining two-argument markers become `f[rad^pow]`. Another grouping rule bundles products of marked values whose numerical real and imaginary parts are both nonzero. The wrapper collects distinct `f[value]` occurrences and passes their contents to `func`, defaulting to `denest11`.

2.3 Single-level and all-level operation

In the single-level mode, the collected markers are replaced with solver results and the expression is simplified. The relevant limitation is the traversal and matching behavior of the replacement rules; the name does not mean that exactly one mathematical root is removed.

In all-level mode, the wrapper maintains a mixed list of unresolved markers and resolved replacement rules. It scans for a marker containing no nested marker, solves it, and propagates its replacement into the remaining unresolved entries. This is a hand-written, bottom-up dependency resolution procedure. For an ordinary finite, acyclic marked expression and a solver that returns marker-free values, it has a natural progress measure: the number of unresolved markers decreases. The code does not make that invariant explicit or check that a custom solver respects it.

The final `ReplaceRepeated` reassembles the result after inner replacements have made outer rule left-hand sides match. This explains why simply replacing it with one `ReplaceAll` would not necessarily be a valid repair. A better implementation would represent the dependencies explicitly or perform one controlled bottom-up traversal.

2.4 The core's state records

An attempted multiplier is described by a triple

$$\{m, P_m, d_m\},$$

where P_m is stored as a pure function and d_m is its degree. The one-argument form of `MinimalPolynomial` really does return a pure-function representation; subsequently calling `minpoly[arg1]` is valid, not a syntax error [3].

The local `trymultiplier` returns either

```
{False, {multiplier, minpoly, degree}}
{True,  {multiplier, minpoly, degree}}
{True,  {multiplier, minpoly, degree}, answer}
```

The first Boolean means "progress" in the search, not necessarily "a solution was found." In particular, discovering a worse degree can count as progress when it provides contrast with a previously seen degree.

A ready multiplier batch has the form

```
{{doFullSet, originRecord}, {m1, m2, ...}}
```

and a deferred batch has the form

```
{{False, smallerRecord}, newmultipliers,
 {smallerRecord, largerRecordOrNull, reductionpower}}
```

The scheduler recognizes a deferred batch because its second element is not a list. At evaluation time it applies the stored function to the stored arguments. A `history` list starts with the sentinel `{0, Infinity, Infinity}` and is maintained in descending order of degree; the last record therefore has the smallest recorded degree.

These structures are compact but overloaded. A tag or an association would make it possible to validate records directly instead of inferring their meaning from list lengths, positions, and whether their last element happens to be a list.

2.5 What the nesting helpers measure

For a Boolean predicate w on expression nodes, define its weight to be one when it is true and zero otherwise. The recursion in `nestdepths` computes

$$h(v) = w(v) + \max_{c \text{ an argument of } v} h(c),$$

with the atom case handled separately. The result is an annotated tree of the form

```
{expression, childRecord1, childRecord2, ..., height}
```

The usual predicate recognizes rational `Power` nodes. Thus the measure counts matching nodes on an argument path; it is not ordinary Wolfram expression depth, field-extension degree, or the minimum nesting depth among all equivalent formulas.

`nestdepthsort` rewrites the annotation into node/height pairs, groups equal heights, and sorts the groups from largest to smallest. `maxnestdepth` instead prunes the annotated tree relative to its maximum height and strips enclosing metadata. Those utilities are used to select the reduction exponent and to compute the diagnostic `extrapower`.

The representation retains the original expression at each annotated node while also storing its children. In a long chain that is logically a repeated storage of subexpressions; actual storage sharing is an implementation detail, not a complexity bound supplied by this program. Because lists also encode the metadata, extending the routines to arbitrary list-valued expressions or algebraic objects with internal list structure requires care. A typed tree or a single traversal returning just the required statistics would make the contract and tests substantially clearer.

3 The algebraic idea, stated precisely

3.1 Reduction exponent and trial numbers

Let the original algebraic value be α. The program chooses a positive integer q by taking an LCM of denominators extracted from selected maximum-depth rational power nodes. It sets

$$\rho = \alpha^q, \qquad \texttt{outerpower} = 1/q. \tag{1}$$

The name "outer root" in the diagnostics should not be read as a theorem that this q is the denominator of every outer root in the input. The selection is based on maximum predicate-dependent nesting height, and other branches may have different, shallower rational powers.

For a trial multiplier m, the code constructs

$$\beta_m = (m\rho)^{1/q}, \qquad P_m(X) = \mathrm{MinPoly}_{\mathbb{Q}}(\beta_m). \tag{2}$$

All powers here have the principal-value meaning of Wolfram Language `Power` [2]. Suppose for now that $q \geq 2$, $m \neq 0$, and all quantities are exact algebraic numbers.

Let K be the algebraic coefficient field used by the GCD operation. With `Extension -> Automatic`, Wolfram extends the coefficient field to include algebraic numbers appearing in the polynomial arguments [4]. Consequently $m\rho \in K$. The field need not be the smallest field $\mathbb{Q}(m\rho)$: its generators can depend on the explicit representation of the coefficients.

The critical computation is

$$G_m(X) = \gcd_{K[X]}\big(P_m(X), X^q - m\rho\big). \tag{3}$$

Proposition 3.1 (Why the GCD is nonconstant)**.** *Under the preceding hypotheses,* $1 \leq \deg G_m \leq q$.

Proof. The number β_m is a root of both polynomials. Its minimal polynomial over K therefore divides both in $K[X]$, and hence divides their GCD. The GCD has positive degree and divides the degree-q binomial, which gives the upper bound. □

Thus, a constant GCD in a valid, exact execution is not ordinary evidence that the multiplier was unhelpful. It indicates a violated assumption, an unresolved computation, or an implementation problem. The source prints a diagnostic in the special case that the returned expression is literally `1`, but does not propagate a structured failure.

Proposition 3.2 (Candidate orbit)**.** *If r is any root of G_m, then*

$$\gamma_{r,j} = \frac{r}{m^{1/q}}\zeta_q^j, \qquad \zeta_q = e^{2\pi i/q}, \quad 0 \leq j < q, \tag{4}$$

satisfies $\gamma_{r,j}^q = \rho$. If $\rho \neq 0$, this orbit contains every qth root of ρ, including the original α.

Proof. Because G_m divides $X^q - m\rho$, one has $r^q = m\rho$. Also $(m^{1/q})^q = m$, even for complex m, since the outer exponent is the integer q. Taking the qth power of (4) therefore gives ρ. If $\rho \neq 0$, the q orbit elements are distinct; the binomial $X^q - \rho$ has exactly q distinct roots, so the orbit contains all of them. For $\rho = 0$ all candidates collapse to zero, which is the only root. □

The roots-of-unity enumeration is therefore a sound way to handle the otherwise unsafe division of principal roots. It does not require the generally false identity $(m\rho)^{1/q}/m^{1/q} = \rho^{1/q}$. What is essential is selecting the member that equals *the original target*.

3.2 What a smaller GCD degree does, and does not, prove

If $e = \deg G_m < q$, solving $G_m(X) = 0$ replaces a degree-q equation over K by a lower-degree equation over K. That can produce a useful radical representation. However, the coefficients of G_m may themselves contain deeply nested radicals; solving it may introduce more radicals or implicit `Root` objects. The inequality $e < q$ is therefore a statement about a polynomial over a field, not a proof that the displayed radical nesting depth has decreased.

Likewise, $d_m = \deg P_m$ is a degree over $\mathbb{Q}$, not the degree over K and not a measure of expression size. The program uses several different notions of complexity without formally separating them:

$$d_m, \qquad e, \qquad \text{syntactic radical depth}, \qquad \text{leaf count}, \qquad \text{number of implicit algebraic objects}.$$

They serve different purposes and cannot safely substitute for one another.

4 A complete successful mathematical example

Consider

$$\alpha = \sqrt{5 + 2\sqrt{6}} = \sqrt{2} + \sqrt{3}. \tag{5}$$

The equality follows by squaring the positive right-hand side. Here $q = 2$ and $\rho = 5 + 2\sqrt{6}$. For $m = 1$,

$$P_1(X) = X^4 - 10X^2 + 1,$$

since $(\alpha^2 - 5)^2 = 24$. Over $K = \mathbb{Q}(\sqrt{6})$,

$$P_1(X) = (X^2 - 5 - 2\sqrt{6})(X^2 - 5 + 2\sqrt{6}),$$

so the GCD with $X^2 - \rho$ has degree two. This trial gives no smaller equation.

For $m = 2$,

$$\beta_2 = \sqrt{10 + 4\sqrt{6}} = 2 + \sqrt{6}, \qquad P_2(X) = X^2 - 4X - 2.$$

The GCD becomes linear:

$$G_2(X) = X - 2 - \sqrt{6}.$$

Dividing its root by $\sqrt{2}$ gives

$$\frac{2 + \sqrt{6}}{\sqrt{2}} = \sqrt{2} + \sqrt{3},$$

which is exactly the desired denesting.

The following table gives independently checked legal trials. It is *not* an assertion that the unmodified scheduler visits them in this exact order.

m	d_m	$P_m(X)$	$\deg G_m$
1	4	$X^4 - 10X^2 + 1$	2
$\sqrt{6}$	4	$X^4 - 24X^2 - 6$	2
2	2	$X^2 - 4X - 2$	1
3	2	$X^2 - 6X + 3$	1
$\sqrt{5}$	8	$X^8 - 490X^4 + 25$	2

The discriminant of P_1 is

$$\operatorname{disc}(P_1) = 147456 = 2^{14}3^2.$$

This explains why searching multipliers formed from its discriminant primes can find 2 or 3. It also illustrates why a multiplier can *increase* the minimal-polynomial degree: multiplying by $\sqrt{5}$ produces a degree-eight trial.

A useful distinction emerges. The original and denested expressions in (5) represent the same degree-four number over $\mathbb{Q}$; denesting did not reduce its intrinsic algebraic degree. The multiplier temporarily moved the search to a number of degree two, and dividing out the multiplier restored a simpler representation of the original degree-four number.

5 How the multiplier search works

5.1 Initial guesses from radicand terms

Without a supplied multiplier list, the code expands a rationalized radicand, attempts to remove rational coefficients from its terms, and derives complementary radical factors. For a factor $v = b^{a/b_0}$, the exponent used in the complementary construction is

$$\frac{b_0 - a}{a},$$

which formally produces $b^{(b_0-a)/b_0}$. This is a heuristic way to complete a fractional exponent to an integer. The source prepends the multiplier 1.

Several qualifications are important. The extraction uses `List @@ Expand[...]` as though it always obtained the summands of a `Plus`. But `Apply` replaces a head: on a product it returns factors, on a power it returns the base and exponent, and on an atom it effectively does nothing [11]. Thus the intended term interpretation is shape-dependent. The expression should first be tested for head `Plus` and otherwise treated as one term.

The removal of coefficients and the recombination of powers are proposal-generation heuristics, not established equivalences between multipliers. They can be acceptable in an incomplete search, provided that every final candidate is independently certified against the original target.

5.2 Degree-gated GCD computation

For each multiplier, the minimal polynomial is always computed first. A GCD is computed only when

$$d_m < d_{\text{best}} \quad \text{or} \quad \gcd(d_m, d_{\text{best}}) < d_{\text{best}}.$$

The first comparison is useful as a heuristic because a rational-degree reduction can signal an exploitable relation. The second is an attempt to exploit incomparable reductions in degree.

Neither comparison is a completeness theorem. In particular, an equal-degree trial is skipped after the best degree is known, although its coefficient-field GCD and its resulting expression need not be identical to those of previous trials. The program supplies no argument that all useful denestings must pass this gate. A

redesigned search should treat this comparison as a priority rule, not confuse it with a logically conclusive rejection of a multiplier.

Nor does the numerical GCD of two field degrees establish a common field of that degree. For example, $\mathbb{Q}(2^{1/4})$ and $\mathbb{Q}(3^{1/6})$ have degrees four and six. Their intersection can only have degree one or two. The first field has the unique quadratic subfield $\mathbb{Q}(\sqrt{2})$ and the second the unique quadratic subfield $\mathbb{Q}(\sqrt{3})$, so their intersection is $\mathbb{Q}$, not a degree-two field. Uniqueness here follows because two distinct quadratic subfields would generate a degree-four biquadratic field: that is impossible in a degree-six field, and would make the real pure-quartic field Galois, which it is not. This example is a warning about degree-only inference, not an asserted counterexample to a particular generated multiplier in the attachment.

5.3 Discriminant-based batches

In the usual branch of `newmultipliers`, the code partially factors $\operatorname{disc}(P_m)$, retains factors passing `PrimeQ`, sorts them, trims certain large prime gaps, and takes a prefix. If the chosen distinct primes are $p_1, \dots, p_k$, it forms

$$\left\{ \prod_{i=1}^{k} p_i^{e_i} : 0 \le e_i \le q-1 \right\}. \tag{6}$$

The implementation obtains this set by calling `Divisors[(Times @@ multiplierprimes)^(q - 1)]`. It unions the result with *all* retained discriminant primes, drops the unit, and multiplies the batch by the earlier successful or promising multiplier.

The union detail matters: a prime excluded from the selected prefix can still be tried singly. It is excluded from the multi-prime divisor combinations, not necessarily from every subsequent trial.

`FactorInteger[n, Automatic]` is a documented partial-factorization mode, not an invalid call. It extracts factors that are easy to find and can leave a composite residual [9]. The source explicitly deletes that residual instead of postponing it. Consequently, the strategy can lose the primes inside a hard residual.

Discriminants are reasonable sources of candidates, but the code proves no theorem that these candidates suffice. A polynomial discriminant can also include factors arising from a chosen generating element rather than only intrinsic field ramification. For a monic minimal polynomial of an algebraic integer, the familiar relation is

$$\operatorname{disc}(P) = \operatorname{disc}(K)\,[\mathcal{O}_K : \mathbb{Z}[\beta]]^2.$$

To see the squared index factor, write the power basis in an integral basis with change-of-basis matrix A. The trace-pairing matrix transforms as $A^{\mathsf{T}} T A$, so its determinant is multiplied by $(\det A)^2$, and $|\det A|$ is the index. The source's general minimal polynomials need not be monic, so even this specific interpretation requires hypotheses not imposed by the implementation.

5.4 Batches from incomparable degree records

When neither observed degree divides the other, `newmultipliers` instead proposes the product of the two stored multipliers. It prints a polynomial GCD of their rational minimal polynomials as a diagnostic. Distinct irreducible rational minimal polynomials usually have GCD one; that diagnostic does not itself construct a common algebraic field or justify the proposed product. The product is another reasonable guess, not an established deduction.

5.5 History, scheduling, and stopping

The scheduler can split a partially processed batch and insert a newly generated batch immediately after it, or defer it according to degree and leaf-count comparisons. The initial full batches are allowed to finish so that

their degree statistics can be used to build additional combinations.

After an apparent success, the source may stop immediately if this was effectively the first recorded trial, or if a ratio involving `history[[2,-1]]` and the new degree is at least q. Because history is sorted by degree, `history[[2,-1]]` is the largest finite degree currently recorded, not an immutable baseline degree of the input. An unrelated worse trial can therefore change this heuristic stopping criterion.

The variable `solution` is replaced by the latest accepted answer without comparing it with the preceding solution under an explicit objective. Returning the original expression at the end means only that this search did not retain an answer. It is not a proof that no denesting exists.

6 Critical defect F1: the wrong branch is certified

6.1 The precise change of target

The decisive sequence is

```
radicand = problem^reductionpower;
...
Select[possibilities,
       PossibleZeroQ[# - radicand^outerpower] &]
```

with `outerpower = 1/reductionpower`. Thus the original target α is silently replaced by

$$\widetilde{\alpha} = (\alpha^q)^{1/q}. \tag{7}$$

Taking the integer power in (1) is legitimate. Inverting that operation with a principal root, without retaining a branch label, is not.

Proposition 6.1 (Exactly when reconstruction preserves the value)**.** *For $\alpha \neq 0$ and an integer $q \geq 2$, write $\alpha = re^{i\theta}$ with $\theta \in (-\pi, \pi]$. Then*

$$(\alpha^q)^{1/q} = \alpha e^{-2\pi i k/q}, \quad q\theta - 2\pi k \in (-\pi, \pi]. \tag{8}$$

In particular, the reconstruction equals α exactly when $\theta \in (-\pi/q, \pi/q]$.

Proof. The principal argument of α^q is $q\theta - 2\pi k$ for the unique integer k placing it in $(-\pi, \pi]$. Taking its principal qth root gives $re^{i\theta - 2\pi i k/q}$. In the allowed range of θ, equality with α occurs precisely when $k = 0$, yielding the stated sector. The asymmetry at the two endpoints is inherited from the chosen principal-argument interval. □

A single principal qth root lies in this sector, which explains why many ordinary positive-real examples work. A negative multiple, a product of such roots, or a rational power with numerator greater than one need not lie in it.

6.2 A direct negative-target counterexample

Consider the core call

```
denest11[-Sqrt[5 + 2 Sqrt[6]], {2}]
```

The nested radical is detected, $q = 2$, and the radicand becomes $5 + 2\sqrt{6}$. The multiplier 2 produces exactly the linear GCD from Section 4. The candidate filter keeps $\sqrt{2} + \sqrt{3}$, since that equals the principal square root of the radicand. But the original target is its negative.

This counterexample concerns `denest11` as defined. It should not be misrepresented as saying that `Strad[-Sqrt[...]]` must lose the same sign: the outer wrapper may leave a simple minus sign outside its marked radical and thereby preserve it. The next two examples explain why the defect nevertheless matters to the wrapper's supported-looking inputs.

6.3 A single rational power with numerator three

Let

$$z = -1 + 2i\sqrt{2}, \qquad u = 1 + i\sqrt{2}.$$

Since $u^2 = z$ and u lies in the right half-plane, u is the principal square root of z. Therefore

$$\alpha = z^{3/2} = u^3 = -5 + i\sqrt{2}. \tag{9}$$

The principal argument of z is between $\pi/2$ and $2\pi/3$, so this is also consistent directly with the principal logarithm definition of $z^{3/2}$. Squaring gives

$$\rho = \alpha^2 = 23 - 10i\sqrt{2}.$$

Its principal square root is $5 - i\sqrt{2} = -\alpha$, not α.

With $m = 1$, the source's trial number is $\beta = 5 - i\sqrt{2}$, whose rational minimal polynomial is

$$P(X) = X^2 - 10X + 27.$$

Over a coefficient field containing $i\sqrt{2}$,

$$\gcd(P(X), X^2 - \rho) = X - 5 + i\sqrt{2}.$$

Thus the very first forced trial can produce a linear GCD and select the wrong branch. The corresponding core regression is

```
denest11[(-1 + 2 I Sqrt[2])^(3/2), {1}]
```

The wrapper pattern accepts any rational exponent, not merely a numerator-one root, so this is not an exotic exponent outside its pattern syntax.

6.4 A bundled product of principal square roots

Define

$$z_1 = -1 + 2i\sqrt{2} = (1 + i\sqrt{2})^2, \qquad z_2 = -2 + 2i\sqrt{3} = (1 + i\sqrt{3})^2.$$

Both displayed square roots are principal. Their product is

$$\alpha = \sqrt{z_1}\sqrt{z_2} = 1 - \sqrt{6} + i(\sqrt{2} + \sqrt{3}), \tag{10}$$

whose real part is negative and imaginary part positive. The wrapper's two-marker product rule can send this whole product to the core, because both radicands contain powers.

For $q = 2$, the reconstructed principal root is again $-\alpha$. The independently checked minimal polynomial of $-\alpha$ is

$$P(X) = X^4 + 4X^3 + 4X^2 + 48X + 144,$$

and its GCD with $X^2 - \alpha^2$ over the explicit algebraic coefficient field is $X + \alpha$. The polynomial step is correct; it is solving for the wrong selected branch.

An integration regression that isolates the first multiplier is

```
DenestRadicals3[
 Sqrt[-1 + 2 I Sqrt[2]] Sqrt[-2 + 2 I Sqrt[3]],
 False,
 Function[t, denest11[t, {1}]]
]
```

The exact final representation of an actual run depends on the preprocessing path, so this test is supplied as an unexecuted integration regression. The algebraic failure of the reconstructed target is independently established.

6.5 The minimal repair

The original target must be an explicit argument to the candidate-selection step:

```
Select[possibilities, ExactAlgebraicEqualQ[#, original] &]
```

The existing roots-of-unity orbit already contains the original target under the hypotheses of Section 3. There is no need to guess a phase from floating-point arguments. A final exact check of the fully reconstructed output against the original input should be a second, independent boundary condition.

7 Critical defect F2: branch-unsafe custom factorization

7.1 The custom representation

`fullfactor` begins with `FactorList`, then converts entries into nested lists representing products, powers, and sums. A typical leaf factor has the form `{base, exponent}` at one stage; deeper lists encode factored summands. `Factorc` reconstructs expressions through repeated patterns that turn pairs into powers, collections into products, and a different nesting shape into sums.

The representation has no distinct heads such as `ProductNode`, `PowerNode`, and `SumNode`. Instead, meaning is inferred from list depth and cardinality. This makes the correctness of its transformations difficult to state, and allows expression data and structural metadata to look alike.

7.2 Two transformations requiring missing hypotheses

The power case in `fullfactor` multiplies the exponent of a power base by the exponent stored in its factor record. Algebraically, it reconstructs

$$(u^a)^b \longmapsto u^{ab}.$$

The common-factor extraction in `combine` moves common bases outside an outer power. If a sum has been written FS, the decoded result has the form

$$(FS)^b \longmapsto F^b S^b.$$

Both are valid under appropriate restrictions, but not for arbitrary principal complex powers. Wolfram's own `Power` only performs the corresponding general automatic transformations when the outer exponent is an integer; unrestricted power expansion requires additional assumptions [2, 14].

Some initial `FactorList` exponents are integers, so not every application of the first rule is wrong. The problem is that repeated unwrapping and common-factor extraction also operate with rational outer exponents and do not preserve the branch conditions required at that point.

7.3 A symbolic counterexample to common-factor extraction

The factor records for $-x$ and $-y$ share the factor -1. When `combine` extracts it from an outer square root, its reconstruction is

$$\sqrt{-x-y} \longmapsto i\sqrt{x+y}. \tag{11}$$

At $x = y = -1$, the left side is $\sqrt{2}$, while the right side is $-\sqrt{2}$. Thus the transformation encoded by the factor extraction cannot be an unconditional identity.

The corresponding Wolfram regression intentionally performs substitution *after* symbolic preprocessing:

```
original = Sqrt[-x - y];
changed = Factorc[original];
RootReduce[(changed - original) /. {x -> -1, y -> -1}]
```

A correct unconditional normalizer must produce zero. Substituting $x = y = -1$ *before* calling `Factorc` would instead feed it the already evaluated number $\sqrt{2}$ and would not test the same transformation. The exact wrapper output of this regression is not claimed as an observed kernel result; the branch-unsafety of (11) is a proved counterexample to the rule.

7.4 Why the wrapper's algebraicity test does not protect it

The first call is `Factorc[expr]`, outside and before the replacement pattern

```
radicand_?AlgebraicQ^pow_Rational
```

Even a symbolic power that is subsequently excluded from algebraic denesting has already passed through custom factorization. Consequently, the wrapper cannot rely on the later pattern test to justify unconditional transformations of the whole input.

Moreover, `ComplexExpand` assumes variables real unless they are specified otherwise [13]. That default is harmless for suitable explicit numeric algebraic inputs, but it is not a replacement for a documented domain contract when symbolic values reach this layer.

7.5 A safer approach

Preserve each principal `Power` node until a value-preserving transformation has been certified. Integer outer powers may be distributed freely. Positive-real common factors can be extracted from rational powers, but arbitrary signs and complex phases cannot. A branch-aware expression tree should retain the original power node rather than flattening it into a base/exponent record whose history has been lost.

The simplest initial repair is to remove `Factorc` from the correctness-critical path and use only transformations whose applicability is established for the exact input. Search quality may temporarily decrease, but a correct unchanged expression is better than a branch-changed "simplification."

8 Definite implementation defects and unsafe failure paths

8.1 F3: The prime comparator ignores its second operand

The source contains

```
#1[[1]] <= #[[2]] &
```

Here `##` is exactly `##1`, not `##2` [7]. The comparator is therefore equivalent to

```
Function[{left, right}, left[[1]] <= left[[2]]]
```

For records `{2, 1}`, `{3, 1}`, it returns `False` in both directions, because $2 \not\leq 1$ and $3 \not\leq 1$. It compares each prime with its own exponent, not with the other prime. Prime-gap trimming and prefix selection downstream therefore operate without the intended numerical ordering invariant.

Replace it with

```
SortBy[factors, First]
```

or, minimally,

```
Sort[factors, #1[[1]] <= #2[[1]] &]
```

Using `<=` is not the defect: Wolfram permits such non-strict ordering functions, and its documentation explicitly discusses `GreaterEqual` for preserving the relative order of equal elements [8]. Importing a C++ strict-weak-ordering objection here would miss the actual problem.

8.2 F4: A stale length is used after deleting pruned primes

The source assigns zeros to selected prime entries and then evaluates

```
multiplierprimes = DeleteCases[multiplierprimes, 0][[
 1 ;; Min[desiredCount, Length[multiplierprimes]]]];
```

The first part of the right-hand side has a shorter list, but the `Length` inside the span still refers to the old value of `multiplierprimes`. The assignment has not happened yet.

For a concrete source-invariant fixture, take the sorted primes

$$\{2, 3, 5, 7, 11, 2003\}, \qquad q = 2.$$

The last ratio is greater than 100, so the loop replaces 2003 by zero and stops at its minimum of five entries. The deleted list has length five. However, `desiredCount` is ten and the old length is six, so the code requests the span `1 ;; 6` of a five-element list. This is an out-of-range part request. The fixture consists entirely of primes; it does not rely on a composite leftover.

The repair is to make the intermediate state explicit:

```
multiplierprimes = DeleteCases[multiplierprimes, 0];
count = Min[desiredCount, Length[multiplierprimes]];
multiplierprimes = Take[multiplierprimes, count];
```

The faulty comparator can affect whether a particular real input reaches this fixture. It does not invalidate the independent indexing defect; fixing the sort alone is insufficient.

8.3 F5: Empty candidate sets and invalid multipliers are unchecked

After solving and filtering, the source tests only whether the result has more than one element. It then unconditionally evaluates `result[[1]]`. An empty list therefore enters a `Part` failure path and can subsequently be packaged as if it were an answer. The outer scheduler further identifies an answer by checking whether the last element of the attempt record is not a list, which is no substitute for validating a successful result type.

The forced list is accepted without checking its elements. In particular, with $m = 0$, $q = 2$, and a nonzero target, the trial minimal polynomial is X and the GCD with X^2 is X. The code then tries to form $0/0$ while

dividing the root by $m^{1/q}$. Candidate filtering cannot repair that invalid arithmetic. A direct failure regression is

```
denest11[Sqrt[5 + 2 Sqrt[6]], {0}]
```

Other invalid inputs include inexact, symbolic, nonalgebraic, and nonfinite multipliers. They can leave `MinimalPolynomial`, `PolynomialGCD`, or `Solve` unevaluated, after which positional indexing propagates malformed state.

Validate exact algebraicity, finiteness, nonzero multipliers, and an integer $q \geq 2$ before the trial. Check the return types and polynomial degrees of each algebraic operation. If no certified candidate remains, return a distinct `NoCandidate` status rather than indexing the list.

8.4 F6: The verification boundary is insufficient

The default `PossibleZeroQ` is documented as a quick symbolic/numerical test that is not always accurate. Current documentation provides `Method -> "ExactAlgebraics"` for guaranteed methods on explicit algebraic numbers [5]. The source does not request that method, does not first establish a precise exact-input contract, and—more decisively—passes the wrong target to the test.

This is not a claim that a specific valid exact candidate is known to be misclassified by the default method in this attachment. The demonstrable defect is that the implementation lacks an explicit exact certification boundary, while F1 shows that even a perfect zero test is being asked the wrong question.

For explicit exact algebraic values, a conservative equality predicate is

```
ExactAlgebraicEqualQ[a_, b_] :=
 Quiet[Check[RootReduce[a - b] === 0, False]]
```

provided the inputs have first passed an exact-algebraic validator. `RootReduce` is designed to reduce algebraic combinations to a single exact algebraic representation [6]. An unresolved result is a failure to certify, not permission to accept a numerical near-zero.

8.5 F9: The input gate and the subsequent algorithm disagree

The initial nested-power detector matches arbitrary `Power` expressions, including integer and symbolic exponents. The later reduction logic selects only `Power[_, _Rational]`, and `trymultiplier` is defined only when the outer power has the form `Rational[1, _Integer]`. Thus passing the first detector does not imply that the reduction exponent is well formed or that the local trial function will evaluate.

The inner `MemberQ[..., Infinity]` also excludes the base expression itself: `Infinity` means levels 1 through infinity, whereas `{0, Infinity}` includes the root [12]. A power directly used as another power's base is therefore not tested in the same way as a power inside a sum in that base. Automatic evaluation may simplify many such cases first, so this is a structural coverage issue, not a claim that every nested-power input is missed.

The helper `AlgebraicQ` is a domain-membership query, not an exactness validator. Official documentation describes immediate recognition of explicit algebraic constructions and permits unresolved membership questions for other expressions [15]. The source adds no independent exact-precision or finite-value check.

The repair is one explicit eligibility predicate, shared by detection, depth measurement, reduction selection, and the solver. Purely symbolic expressions should either be excluded from the numeric core or handled in a separate, assumption-aware interface.

8.6 F10: Loading the file changes global kernel state

The top of the source executes `Unprotect[Print]` and never restores its previous protection status. The `Block` in `Strad` localizes the value used for `Print`; it does not undo that top-level mutation of attributes. This changes the surrounding kernel merely by loading the file.

More generally, `Block` uses dynamic scoping, while `Module` introduces fresh symbols [10]. The many dynamically localized names are not automatically bugs: the source does localize the parent loop variables, including `i3`. But private syntax markers, scratch variables, and callbacks are easier to isolate using a package context and fresh lexical symbols. Deliberate dynamic settings such as assumptions or logging configuration are separate concerns.

Replace global print suppression with an explicit logging function or a `Verbose` option. Do not unprotect a system symbol as a permanent load-time side effect. A package should also avoid `ClearAll` on generic global names such as `combine` that may already belong to the caller's session.

9 Search limitations, resource risks, and maintainability

9.1 F7: the multiplier cap can be exceeded by orders of magnitude

With k distinct selected primes, the divisor set in (6) has exactly q^k elements: each of the k exponents independently has q choices. The source chooses a prefix length resembling

$$k = \min\left(L, \max\left(5, \left\lceil \frac{\log 1000}{\log q} \right\rceil\right)\right),$$

where L is the available prime count, subject to the separate stale-length bug. For sufficiently many available primes, the number of nonunit divisor candidates is therefore:

q	Selected k	$q^k - 1$
2	10	1,023
3	7	2,186
10	5	99,999
100	5	9,999,999,999

These counts omit extra singleton primes introduced by the union. The constant `multipliercap = 1000` is therefore neither a strict bound on trials nor a bound on allocated memory. The forced minimum of five primes is particularly hazardous for high-order roots.

To bound the complete divisor product alone by M nonunit candidates, one needs $q^k - 1 \le M$, hence

$$k \le \left\lfloor \log_q(M+1) \right\rfloor.$$

Even that correction is insufficient if another union adds more candidates, or if subsequent batches have no global budget. A robust scheduler must impose a total trial limit and generate candidates lazily rather than allocating the entire cartesian product first.

9.2 Exponential subset expansion after the initial pass

The post-pass construction uses each selected multiplier to the powers 0 and 1 and distributes across the choices. For r selected multipliers, this creates up to 2^r combinations before duplicate elimination and simplification. Removing duplicates afterward does not prevent the initial allocation.

Its simplifications strip integer factors and reduce some exponents modulo their denominators. These operations are only heuristics on the proposed multiplier set. In general, multiplying a multiplier by an arbitrary integer is not an algebraically irrelevant change: Section 4 shows that $m = 1$ and $m = 2$ have different trial degrees. Normalization modulo certified qth powers would require its own justification and branch handling.

9.3 No global cache, budget, or explicit exhaustion state

There is no global set of already attempted exact multiplier values, no minimal- polynomial cache, no total candidate budget, no degree or coefficient-size limit, and no controlled time or memory budget around expensive algebraic operations. The same multiplier can arise through different discriminant batches or products. Syntactic `DeleteDuplicates` within one list is not an algebraic cache across the search.

The lack of these controls does not, by itself, prove that a particular valid input will loop forever. It does mean that the program offers no application-level bound on work, no explicit proof of termination of its changing work queue, and no clean distinction between "budget exhausted" and "no candidate exists." A user-facing implementation should expose those distinctions directly.

9.4 Degree and coefficient growth

Computing a minimal polynomial of a radical combination can require a compositum of several algebraic extensions. Both its degree and the size of its coefficients can grow rapidly as radicals and multipliers are combined. The following GCD may work over another large explicit coefficient field, and discriminant factorization adds integer-arithmetic costs unrelated to the visual size of the original input.

A useful cost model is therefore not just the number of multiplier trials, but a sum of costs depending on rational degree, coefficient-field degree, coefficient height, root order, and candidate representation size. The attachment provides no measurements from which reliable wall-clock estimates can be inferred. In particular, the number `1000` should not be interpreted as an experimentally validated performance bound.

9.5 F8: success is not measured against a denesting objective

The comment "progress because a denesting occurred" follows only a smaller GCD degree and a surviving equality test. There is no comparison of the answer's radical depth with the input's, no prohibition on introducing `Root`, and no comparison with a previously retained answer.

An explicit objective could first exclude new implicit algebraic objects and then lexicographically minimize

$$(\text{radical depth}, \text{radical-node count}, \text{leaf count}).$$

Alternatively, a user might prefer minimum leaf count even at the cost of a slightly greater nesting depth. Either policy is defensible; silently substituting minimal- polynomial degree for both is not.

9.6 Forced multipliers are not necessarily the only multipliers

A list supplied as the second argument to `denest11` replaces the initial multiplier list and changes its batch flag. It does not disable every path that calls `newmultipliers`.

The worked example supplies a concrete control-flow illustration. With `{1, Sqrt[5]}`, the first trial has degree four and no smaller GCD; the second has degree eight. While no worse degree is yet known, that worse trial counts as progress. The subsequent history logic can schedule new discriminant-based multipliers. Thus an option called "forced multipliers" is better described as "initial multipliers" unless an additional option explicitly forbids generation.

9.7 Numerical tests should not be blamed for the wrong reason

The real-part sign test uses 100-digit numerical evaluation; the complex grouping rule uses 10-digit numerical evaluation. A fixed precision is not an exact sign or zero certificate for an arbitrarily ill-conditioned algebraic expression. Nevertheless, the sign-normalization loop negates both a factor and an accumulated sign, preserving their product regardless of which branch of that test executes. Similarly, grouping existing factors into a marked product does not itself change their mathematical product.

The immediate risks here are representation sensitivity, inconsistent search paths, and poor behavior near cancellation. The decisive demonstrated wrong-value bug is the later principal-root reconstruction, not an unsupported claim that every numerical sign misclassification changes the product's value.

9.8 F12 and other maintenance issues

`extrapower` is computed from inner nesting levels and printed, but it is never used to construct a trial, alter q, or improve a result. The diagnostic "extra root" should not be taken as evidence that an extra-root algorithm is implemented. `maxprimes = 14` is likewise unused. Other local names are unused or used only for diagnostics, and obsolete alternatives remain commented out.

`fullfactor` stops its local refinement loop after five passes whether or not `done` is true. This is an arbitrary normalization limit, not a documented mathematical bound. A limit can be a legitimate safeguard, but reaching it should be explicit and should preserve a well-defined representation rather than silently suggest completion.

The helper `combine` removes unit-base entries only inside a loop ending at `Length[base]-1`; a final or lone unit entry can survive that cleanup. This is primarily an inconsistency in canonicalization, not on its own a proved wrong-value case. The subsequent recursive `Position` and `Delete` operations also rely on the private list encoding not containing look-alike data.

Finally, undocumented positional records, a shared string tag for `Catch`/`Throw`, diagnostic printing embedded in predicates, and global helper definitions make individual components difficult to test independently. These are engineering shortcomings distinct from the mathematical counterexamples.

10 The denominator rationalizer is mathematically sound

This helper deserves a positive assessment, subject to its domain assumptions. For a nonzero algebraic denominator d, let

$$P(X) = a_n X^n + \cdots + a_1 X + a_0$$

be its minimal polynomial over $\mathbb{Q}$. Because $d \neq 0$, one has $a_0 \neq 0$. Polynomial division gives

$$P(X) = (X - d)Q(X),$$

with coefficients of Q in $\mathbb{Q}(d)$. Evaluating at $X = 0$ gives

$$a_0 = -dQ(0), \qquad \frac{1}{d} = -\frac{Q(0)}{a_0}.$$

That is exactly the formula implemented by

```
PolynomialQuotient[minpoly[x], x - denom, x] /. x -> 0
```

followed by division by `-minpoly[0]` and multiplication by the numerator.

Equivalently, the annihilating relation $P(d) = 0$ yields

$$\frac{1}{d} = -\frac{a_1 + a_2 d + \cdots + a_n d^{n-1}}{a_0}. \tag{12}$$

The stored variable `conjugate` is a polynomial, not the ordinary complex conjugate of d; the name is misleading, but the formula is valid.

For $d = \sqrt{2} + \sqrt{3}$, the polynomial is $X^4 - 10X^2 + 1$, so

$$\frac{1}{d} = 10d - d^3 = \sqrt{3} - \sqrt{2}.$$

For $d = 1 + i\sqrt{2}$, the polynomial is $X^2 - 2X + 3$, giving

$$\frac{1}{d} = \frac{2 - d}{3} = \frac{1 - i\sqrt{2}}{3}.$$

Both identities, and a rational-denominator case, are checked independently in the companion script.

The missing engineering pieces are explicit rejection of zero or unsupported denominators, checked polynomial operations, and a policy about expression growth. Also, a sum of rational expressions need not already have the desired common syntactic denominator. Calling `Together` before denominator extraction is a normalization choice; without it the helper can preserve the value while failing to rationalize every denominator hidden in separate summands.

11 A conservative repair and redesign

11.1 First establish a correctness boundary

The first milestone should not be a more aggressive multiplier generator. It should be a kernel with the following invariant:

> Every returned candidate has been proved equal to the original exact algebraic input; search heuristics can change which candidates are found, but cannot change what counts as an acceptable answer.

Concretely, retain the original input, remove branch-unsafe preprocessing, reject invalid multipliers, enumerate the existing root-of-unity orbit, and compare each candidate with that original value using exact algebraic arithmetic. If no candidate is certified, keep the original. At the top-level boundary, certify the reconstructed whole expression again whenever its domain permits exact comparison.

For symbolic host expressions, exact algebraic comparison of the whole expression is not available. There the rewrite engine must preserve the host structure and replace only validated numeric subexpressions, or establish identities under explicit assumptions. It must not silently treat a symbolic identity as a numeric zero-testing problem.

11.2 Give the result an explicit type

A trial should return a record such as

```
<|"Status" -> "Candidate",
  "Original" -> original,
  "Multiplier" -> multiplier,
```

```
  "MinimalPolynomial" -> polynomial,
  "RationalDegree" -> degree,
  "RelativeGCDDegree" -> gcdDegree,
  "Candidates" -> verifiedCandidates|>
```

Other statuses should distinguish absence of a candidate, no GCD-degree reduction, invalid input, failed computation, and an exceeded budget. A search can then record a worsening degree without pretending that this is a successful denesting, and it can return its best certified candidate when a resource budget is reached.

11.3 A replacement candidate kernel

The companion `safe_candidate_kernel.wl` supplies a small, conservative implementation of the mathematically justified trial. It does not reproduce the attachment's scheduler or claim completeness. Its important differences are that q is explicit and validated, the original value is never discarded, a nonzero exact multiplier is required, and an empty candidate set is an ordinary result.

The central extraction and verification are:

```
rho = original^q;
beta = (multiplier rho)^(1/q);
polynomial = MinimalPolynomial[beta, x];
g = PolynomialGCD[polynomial,
                  x^q - multiplier rho,
                  Extension -> Automatic];
roots = x /. Solve[g == 0, x];
candidates = Flatten[Outer[Times,
   roots/multiplier^(1/q),
   Exp[2 Pi I Range[0, q - 1]/q]]];
verified = Select[candidates,
   ExactAlgebraicEqualQ[#, original] &];
```

The complete file adds guards, return-type checks, exact duplicate elimination, and a time-budget wrapper. It is a proposed implementation, not a target-kernel-tested patch. Its algebraic construction is the one proved in Section 3.

11.4 Separate equivalence testing from representation scoring

Do not apply `RootReduce` to the retained representation merely because it is used to prove equivalence. The verification representation and the preferred output representation have different purposes. An exact `Root` object may be excellent for equality but undesirable as the visible result of a radical denester.

For root-free numeric inputs, a defensible cost tuple is

$$\big(\#\text{implicit algebraic objects},\ \mathrm{rdepth},\ \#\text{rational-power nodes},\ \text{leaf count}\big).$$

Count a rational-power node only when its exponent has denominator greater than one. Treat `Root` and `AlgebraicNumber` explicitly; do not accidentally reward them with a radical depth of zero. Use a strict improvement rule and retain the best certified answer found so far.

A single traversal can compute depth and other structural statistics. The current combination of annotated nested lists, repeated rewrites, pruning, and positional extraction is more complicated than necessary and exposes more representation- dependent corner cases.

11.5 Rebuild the scheduler around budgets and a visited set

The search layer should own a priority queue of untried multipliers, a set of canonical exact values already attempted, and a cache of trial results. Generated batches should be lazy streams. Enforce a total trial budget, a maximum candidate representation size, and time and memory budgets around expensive algebraic operations. A hard budget must be checked before materializing a cartesian product, not after it.

Use distinct options for an initial multiplier list and an exclusive multiplier list. Make heuristic prime pruning optional, record discarded composite residuals, and expose the reason each candidate was proposed. These controls make a missed denesting diagnosable rather than turning a heuristic failure into a mysterious "no progress" printout.

11.6 A cheap exact fast path for quadratic surds

Before invoking the general search, recognize a common special case:

$$\sqrt{a+b\sqrt{d}}, \qquad a,b,d\in\mathbb{Q}, \quad d>0.$$

Suppose $a>0$ and

$$\Delta = a^2-b^2d = \delta^2 \geq 0, \qquad \delta\in\mathbb{Q}_{\geq 0}.$$

Set

$$u=\frac{a+\delta}{2}, \qquad v=\frac{a-\delta}{2}.$$

Then $u,v\geq 0$, $u+v=a$, and $4uv=b^2d$. Consequently, for $b\neq 0$,

$$\sqrt{a+b\sqrt{d}} = \sqrt{u}+\operatorname{sgn}(b)\sqrt{v}. \tag{13}$$

Indeed the right side squares to the radicand and is nonnegative because $u\geq v$. The case $b=0$ is simply $\sqrt{a}$. This derivation also identifies the positivity and branch hypotheses that a general symbolic rewrite must not omit.

For $a=5$, $b=2$, and $d=6$, one has $\delta=1$, $u=3$, $v=2$ and immediately obtains (5). Such a fast path avoids minimal polynomials and discriminant factorization for an important family while retaining exact verification.

12 Testing strategy and reproducibility

12.1 What was actually executed

The supplied `independent_checks.py` ran successfully and recorded 26 checks. These cover the classical denesting identity; its minimal polynomial and discriminant; degrees and GCDs for five representative multipliers; the negative, three-halves, and merged-product branch constructions; the denominator inverse formula; and small Python models of the comparator, stale-length, and multiplier- count defects.

The exact Python and SymPy versions, source hash, symbolic results, and per-check statuses are in `verification_results.json`. The human-readable output is in `verification_results.txt`. These files do not assert that `Strad`, `Factorc`, or the complete Wolfram search ran successfully or failed in a particular observed way.

12.2 Wolfram regressions supplied for execution

The companion test file contains assertions for correctness, not assertions that encode the buggy behavior as desirable. Some are therefore expected to fail against the original code. Important cases are:

Test family	Required property
Positive classical radical	The answer equals $\sqrt{2}+\sqrt{3}$.
Negative direct core input	The sign of $-\sqrt{5+2\sqrt{6}}$ is preserved.
Complex exponent $3/2$	The answer equals $-5+i\sqrt{2}$, not its negative.
Bundled complex product	Both the direct core and wrapper preserve the product in (10).
Symbolic factorization	Substitution after normalization does not change the value of $\sqrt{-x-y}$.
Zero multiplier	The trial rejects invalid input or conservatively preserves the original value; it does not return a malformed "answer."
Prime ordering and pruning	The comparator uses both records; deleting primes cannot invalidate a subsequent slice.
Global state	Loading the source does not permanently unprotect `Print`.
All-level operation	The result remains equal to the original nested input under both modes.
Replacement candidate kernel	Certified candidates preserve all tested branches and invalid multipliers return failures.

The runner records the actual kernel version and uses finite evaluation timeouts. It restores the incoming attributes of `Print` after loading and testing the source. The package remains an audit bundle, not a claim that the replacement kernel has completed platform-specific validation.

12.3 Properties beyond individual examples

A mature test suite should enforce semantic preservation on generated exact algebraic inputs, stability under multiplication by roots of unity, and consistency between equivalent input representations. Tests should compare exact values rather than one preferred textual form.

For accepted rewrites, verify both equivalence and strict cost improvement. For unchanged returns, verify the status says only what is known: no improvement found within the selected strategy and budget. Exercise cancellation near the real and imaginary axes, rational exponents with numerator greater than one, negative exponents, mixed root orders, rational denominators, explicit `Root` inputs, invalid forced multipliers, duplicate multipliers, and large prime gaps.

The raw original source, its hash, the independent checker, and the proposed regressions are included so that a future kernel run can distinguish a source change, a language-version change, and a mistaken analysis.

13 Final assessment

The attachment contains a useful experimental idea: a multiplier can lower the rational degree of an auxiliary radical, allowing a polynomial GCD over an algebraic coefficient field to expose a simpler representation. The classical example shows this mechanism exactly, and the denominator rationalizer is supported by a clean algebraic proof.

The principal weakness is not merely that the search is heuristic. It is that normalization, branch selection, proposal generation, and correctness verification are entangled. A heuristic can be incomplete while remaining perfectly reliable; this implementation does not maintain that separation. Its wrong-target filter can approve a sign-reversed number, and its factorization rules can change branches before the search begins.

The practical repair order is therefore: preserve and certify the original value; remove or guard unsafe power transformations; fix the comparator, stale slice, and empty-result paths; define explicit input and output

types; then bound and improve the multiplier search. After that foundation is in place, stronger heuristics and specialized denesting algorithms can be added without jeopardizing correctness.

A Companion files and build instructions

The archive is self-contained. It includes the article source and PDF, the unmodified Wolfram source, the independent checker and its executed results, a conservative replacement candidate kernel, Wolfram regression tests and a runner, and a README.

Compile this article from the archive directory with:

```
pdflatex -interaction=nonstopmode -halt-on-error review.tex
pdflatex -interaction=nonstopmode -halt-on-error review.tex
```

Run the independent checks with a Python installation containing SymPy:

```
python independent_checks.py
```

Run the Wolfram regressions in a kernel environment with:

```
wolframscript -file run_tests.wl
```

The tests intentionally express the desired correctness properties, so the original source is expected to fail several of them. No files containing font binaries are included or required beyond the fonts supplied by a standard suitable TeX installation.

B Reference implementation of one certified candidate trial

This listing is a proposed building block, not a complete replacement for `DenestRadicals3`. It is reproduced from `safe_candidate_kernel.wl`. Its exact-algebraic guards are intentionally conservative: an input whose domain membership cannot be established is rejected rather than guessed.

```
(* Proposed, conservative replacement for ONE multiplier trial.
   This file has not been executed in a Wolfram kernel in this review.
   It is not a complete denester or a completeness claim. *)

BeginPackage["RadicalDenestingAudit`"];
ExactAlgebraicNumberQ::usage =
  "Test for an explicitly known, exact numeric algebraic value.";
ExactAlgebraicEqualQ::usage =
  "Conservatively certify equality of exact algebraic values.";
TryDenestCandidate::usage =
  "TryDenestCandidate[original,q,m] returns verified candidates.";
RadicalDepth::usage =
  "Count the maximum number of rational-power nodes on a path.";
CandidateCost::usage =
  "Return an explicit representation cost, penalizing Root objects.";
BestImprovement::usage =
  "Retain the original unless a certified candidate improves cost.";
Options[TryDenestCandidate] = {"TimeBudget" -> 30};

Begin["`Private`"];

ExactAlgebraicNumberQ[e_] := TrueQ[
  NumericQ[e] && Precision[e] === Infinity &&
```

```
  Quiet[Check[Element[e, Algebraics], False]]
];

ExactAlgebraicEqualQ[a_, b_] :=
  If[ExactAlgebraicNumberQ[a] && ExactAlgebraicNumberQ[b],
    Quiet[Check[RootReduce[a - b] === 0, False]],
    False
  ];

TryDenestCandidate[original_, q_, multiplier_,
    OptionsPattern[]] := Module[{budget = OptionValue["TimeBudget"]},
  If[!(NumericQ[budget] && TrueQ[0 < budget < Infinity]),
    Return[Failure["InvalidBudget", <|"Budget" -> budget|>]]];
  TimeConstrained[
    trialBody[original, q, multiplier], budget,
    Failure["BudgetExceeded", <|"Multiplier" -> multiplier|>]
  ]
];

trialBody[original_, q_, multiplier_] := Module[
  {x, rho, beta, p, g, d, e, rules, roots, candidates, verified,
   metadata},
  If[!IntegerQ[q] || q < 2,
    Return[Failure["InvalidRootOrder", <|"Order" -> q|>]]];
  If[!ExactAlgebraicNumberQ[original] ||
     !ExactAlgebraicNumberQ[multiplier],
    Return[Failure["InvalidInput", <||>]]];
  If[ExactAlgebraicEqualQ[multiplier, 0],
    Return[Failure["ZeroMultiplier", <||>]]];

  rho = original^q;
  beta = (multiplier rho)^(1/q);
  p = Quiet[Check[MinimalPolynomial[beta, x], $Failed]];
  If[p === $Failed || !PolynomialQ[p, x],
    Return[Failure["MinimalPolynomialFailed", <||>]]];
  d = Exponent[p, x];
  If[!IntegerQ[d] || d < 1,
    Return[Failure["InvalidMinimalPolynomialDegree", <||>]]];

  g = Quiet[Check[
    PolynomialGCD[p, x^q - multiplier rho,
                  Extension -> Automatic], $Failed]];
  If[g === $Failed || !PolynomialQ[g, x],
    Return[Failure["PolynomialGCDFailed", <||>]]];
  e = Exponent[g, x];
  If[!IntegerQ[e] || e < 1 || e > q,
    Return[Failure["InconsistentGCDDegree", <|"Degree" -> e|>]]];

  metadata = <|"Original" -> original,
    "Multiplier" -> multiplier, "RootOrder" -> q,
    "MinimalPolynomial" -> Function[Evaluate[p /. x -> Slot[1]]],
    "RationalDegree" -> d, "RelativeGCDDegree" -> e|>;
  If[e == q,
    Return[Join[metadata, <|"Status" -> "NoDegreeReduction",
                            "Candidates" -> {}|>]]];

  rules = Quiet[Check[Solve[g == 0, x], $Failed]];
  If[rules === $Failed || !ListQ[rules],
    Return[Failure["SolveFailed", metadata]]];
  roots = Quiet[Check[x /. rules, $Failed]];
```

```
  If[!ListQ[roots] || !FreeQ[roots, x] ||
     !AllTrue[roots, ExactAlgebraicNumberQ],
    Return[Failure["UnresolvedRoots", metadata]]];

  candidates = Quiet[Check[
    Flatten[Outer[Times, roots/multiplier^(1/q),
      Exp[2 Pi I Range[0, q - 1]/q]]], $Failed]];
  If[!ListQ[candidates],
    Return[Failure["CandidateConstructionFailed", metadata]]];
  verified = DeleteDuplicates[
    Select[candidates, ExactAlgebraicEqualQ[#, original] &],
    ExactAlgebraicEqualQ];
  Join[metadata, <|"Status" ->
    If[verified === {}, "NoCandidate", "Candidate"],
    "Candidates" -> verified|>]
];

RadicalDepth[e_] := If[AtomQ[e], 0,
  If[MatchQ[e, Power[_, _Rational]],
    1 + RadicalDepth[e[[1]]],
    Max[Prepend[RadicalDepth /@ (List @@ e), 0]]
  ]
];

CandidateCost[e_] := {
  Count[e, _Root | _AlgebraicNumber, {0, Infinity}],
  RadicalDepth[e],
  Count[e, Power[_, _Rational], {0, Infinity}],
  LeafCount[e]
};

BestImprovement[original_, candidates_List] := Module[
  {valid, best, oldCost, newCost},
  If[!ExactAlgebraicNumberQ[original],
    Return[Failure["InvalidInput", <||>]]];
  valid = Select[candidates, ExactAlgebraicEqualQ[#, original] &];
  best = First[SortBy[Prepend[valid, original], CandidateCost]];
  oldCost = CandidateCost[original];
  newCost = CandidateCost[best];
  If[newCost =!= oldCost && OrderedQ[{newCost, oldCost}],
    best, original]
];

End[];
EndPackage[];
```

C The complete original source, with line numbers

The following listing is read directly from the byte-preserved `source.wl`. Long physical lines may wrap visually, but the printed line numbers refer to the original physical lines. No repairs have been applied to this listing.

```
ClearAll[Strad];
Unprotect[Print];
Strad[eaf__] := Block[{Print = (Null &)}, DenestRadicals3[eaf]];
ClearAll[DenestRadicals3];
DenestRadicals3[expr_, alllevels_ : False, func_ : denest11] :=
 Block[{f, mod, level, problems, i1, i2},
  If[alllevels =!= True,
   level = ReplaceAll,
   level = ReplaceRepeated
   ];
  mod = level[Factorc[expr],
    radicand_?AlgebraicQ^pow_Rational :> Block[{factored, SumQ, i, j},
      SumQ[expression_] :=
       MemberQ[
        expression, _Plus | Complex[Except[0], _], {0, Infinity}];
      factored = FactorTermsList[radicand];
      factored[[2]] =
       Flatten[{Replace[Factorc[factored[[2]]],
          prod_Times :> List @@ prod]}];
      factored[[1]] =
       DeleteCases[{#, ComplexExpand[factored[[1]]/#]} &[
         GCD[Re[factored[[1]]], Im[factored[[1]]]]], 1];
      factored[[1]] = #[[1]]^#[[2]] & /@
        Flatten[FactorInteger /@ factored[[1]], 1];
      factored = Flatten[factored];
      factored =
       factored //.
        list : {a___, expr1_, b___, expr2_, c___} :> {a, b, c,
           ComplexExpand[expr1*expr2]} /; And[SumQ[expr1], SumQ[expr2]];
      j = 1;
      For[{i = 1}, i <= Length[factored], i++,
       If[N[Re[factored[[i]]], 100] < 0,
        factored[[i]] = -factored[[i]];
        j *= -1
        ]
       ];
      factored = DeleteCases[Prepend[factored, j], 1];
      f[Simplify[
        Times @@ (Simplify[RationalizeDenominator[Together[#]]] & /@
           factored)], pow]
      ]
    ];
  mod = mod //.
    prod : Times[a___, expr1 : f[rad1_, pow1_], b___,
       expr2 : f[rad2_, pow2_], c___] :>
     Times[a, b, c, f[(rad1^pow1)*(rad2^pow2)]] /;
      And[! FreeQ[rad1, Power], ! FreeQ[rad2, Power]];
  mod = level[mod, f[rad_, pow_] :> f[rad^pow]];
  mod = mod //.
    prod : Times[a___, expr1 : f[rad1_], b___, expr2 : f[rad2_],
       c___] :>
     Times[a, b, c, f[rad1*rad2]] /;
      And[And[Re[#] != 0, Im[#] != 0] &[N[rad1, 10]],
       And[Re[#] != 0, Im[#] != 0] &[N[rad2, 10]]];
  problems = DeleteDuplicates[Cases[mod, f[_], {0, Infinity}]];
  Print[problems];
  Print[];
  If[alllevels =!= True,
   mod =
    Simplify[
     mod /. ((# -> Replace[#, f[val_] :> func[val]]) & /@ problems)],
   For[{i1 = 1}, MemberQ[problems, f[__]] === True, i1++,
```

```
    i1 = Mod[i1, Length[problems], 1];
    If[MatchQ[problems[[i1]], f[problem__] /; FreeQ[problem, f[__]]],
     problems[[i1]] = problems[[i1]] -> (problems[[i1]] /. f -> func);
     For[{i2 = 1}, i2 <= Length[problems], i2++,
      If[Head[problems[[i2]]] =!= Rule,
       problems[[i2]] = problems[[i2]] //. problems[[i1]];
       ]
      ];
     ];
    ];
   mod = Simplify[mod //. problems];
   ];
  Return[mod];
  ]

ClearAll[denest11];
denest11[problem_, forcedmultipliers_ : 0] :=
 Block[{radicand, outerpower, reductionpower, extrapower,
   multiplierstack, multipliers, multiplier, terms, history, attempt,
   solution, updatehistory, stackchange, addmultipliers,
   trymultiplier, newmultipliers, x, i1, i2, i3},
  If[! MemberQ[problem,
     Power[inner_?(Function[var, MemberQ[var, Power[a_, b_], Infinity]]),
       outer_], {0, Infinity}],
   Return[problem];
   ];
  Print["problem  ", problem];
  reductionpower = (LCM @@ (maxnestdepth[problem,
        MatchQ[#, Power[_, _Rational]] &] /. {Power[base_, exp_], _} :>
         Denominator[exp]));
  outerpower = 1/reductionpower;
  radicand = problem^reductionpower;
  extrapower =
   Times @@ ((LCM @@ #) & /@ (DeleteCases[
         Cases[nestdepthsort[radicand,
           MatchQ[#, Power[_, _Rational]] &],
          patt : {{expr_, nest_}, Repeated[{_, _}, {0, Infinity}]} /;
           nest >= 2], {Except[Power[_, _Rational]], _}, {2}] /.
        patt : {Power[base_, exp_], nest_} :> Denominator[exp]));
  Print["outer root: ", reductionpower, "  extra root: ", extrapower];
  Print[];

  (*tries a multiplier
  returns {<boolean progress>, {multiplier, minpoly, degree}, [answer]}*)
  trymultiplier[radicand_, outerpower_, multiplier_, best_,
     worseknown_] /; MatchQ[outerpower, Rational[1, _Integer]] :=
   Block[{minpoly, degree, polygcd, possibilities, result, expected,
     arg1},
    minpoly = MinimalPolynomial[(multiplier*radicand)^outerpower];
    degree = Exponent[minpoly[arg1], arg1];
    result = {False, {multiplier, minpoly, degree}};
    Print["multiplier: ", multiplier, "   minpoly degree: ", degree];
    If[Or[degree < best, GCD[degree, best] < best],
     polygcd =
      PolynomialGCD[minpoly[arg1],
       arg1^(1/outerpower) - multiplier*radicand,
       Extension -> Automatic];
     If[And[Exponent[polygcd, arg1] < (1/outerpower), polygcd =!= 1],
      possibilities =
       Flatten[Outer[Times,
         arg1/(multiplier^outerpower) /. Solve[polygcd == 0, arg1],
         Exp[2*I*\[Pi]*Range[0, (1/outerpower) - 1, 1]/(1/outerpower)]]];
      result =
       DeleteDuplicates[
        Select[possibilities, PossibleZeroQ[# - radicand^outerpower] &]];
      If[Length[result] > 1,
       Print[Length[result], " possible solutions, taking the first"];
       Print[result];
       result = Take[result, 1];
```

```
        ];
       (*progress because a denesting occurred*)
       result = {True, {multiplier, minpoly, degree}, result[[1]]},
       If[polygcd =!= 1,
        If[
         Or[degree < best,
          And[GCD[degree, best] < best, worseknown =!= True]],
         (*progress because degree of minpoly has decreased*)
         result = {True, {multiplier, minpoly, degree}}
         ],
        Print[
         "PolynomialGCD error: ", {radicand, outerpower, multiplier,
          minpoly}]
        ]
       ],
      If[And[worseknown =!= True, degree != best],
       (*progress because this multiplier has greater degree than a pre\
vious multiplier*)
       Print["previous progress identified"];
       result = {True, {multiplier, minpoly, degree}};
       ]
      ];
     Return[result];
     ];

   (*form for attempts is {multiplier, minpoly, degree},
   attempt2 is Null if only one attempt is available*)
   newmultipliers[smaller_, larger_, reductionpower_] :=
    Block[{done, new, primes, multiplierprimes, minprimes, maxprimes,
      primeratio, multipliercap, arg1},
     minprimes = 5;
     maxprimes = 14;
     primeratio = 100;
     multipliercap = 1000;

     (*two possibilites: reduction in degree,
     reduction only in GCD of degrees*)
     If[Or[larger === Null,
       GCD[smaller[[-1]], larger[[-1]]] ==
        Min[smaller[[-1]], larger[[-1]]]],
      (*one degree is a multiple of the other, usual case*)
      primes =
       DeleteCases[
        FactorInteger[Discriminant[smaller[[2]][arg1], arg1],
         Automatic], {-1, 1}];
      primes =
       Sort[DeleteCases[primes,
         factor : {prime_, _} /;
          And[! PrimeQ[prime], (Print["composite factor: ", factor];
            True)]], #1[[1]] <= #[[2]] &];
      primes = Flatten[primes[[All, 1]]];
      multiplierprimes = primes;
      For[{i3 = Length[primes]},
       And[i3 > minprimes,
        multiplierprimes[[i3]]/multiplierprimes[[i3 - 1]] > primeratio],
        i3--,
       multiplierprimes[[i3]] = 0;
       ];
      multiplierprimes =
       DeleteCases[multiplierprimes, 0][[
        1 ;; Min[
          Max[minprimes, Ceiling[Log[multipliercap]/Log[reductionpower]]],
           Length[multiplierprimes]]]];
      new =
       smaller[[1]]*
        Drop[Union[
          Divisors[(Times @@ multiplierprimes)^(reductionpower - 1)],
          primes], 1];
      ,
```

```
      (*different degree reductions in each degree from a larger degree\
*)
      Print[
       "GCD of smallest degrees is smaller than any observed degree: \
", {smaller[[{1, -1}]], larger[[{1, -1}]]}];
      new = smaller[[1]]*larger[[1]]*{1};
      Print[PolynomialGCD @@ {smaller[[2]][x], larger[[2]][x]}];
      ];
     new = {{False, smaller}, ComplexitySort[new]};
     Return[new];
     ];

   (*compute initial set of multipliers*)
   If[ListQ[forcedmultipliers],
    multipliers = {{False, {0, Infinity, Infinity}}, forcedmultipliers},
    (*old, may be better*)
    (*
    terms=DeleteCases[Replace[#,{Times[a___,
    Alternatives[_Rational,_Integer],b___]\[RuleDelayed]a*b,
    Alternatives[_Rational,_Integer]\[Rule]0,Complex[a_,
    b_]\[RuleDelayed]Sign[b]*I}]&/@(List@@Expand[
    RationalizeDenominator[radicand]]),0];
    *)
    terms =
     DeleteCases[
      Replace[#, {Times[a___, Alternatives[_Rational, _Integer],
            b___] :> a*b, Alternatives[_Rational, _Integer] -> 0,
          Complex[a_, b_] :> Sign[b]*I}] & /@ (List @@
         Expand[RationalizeDenominator[radicand]]), 0];
    multipliers = Prepend[Replace[If[Head[#] === Times,

          Times @@ (Function[{var},
              var^(maxnestdepth[var,
                   Function[{var2},
                    MatchQ[var2, Power[_, _Rational]]]][[
                  1]] /. {{Power[base_,
                     exp_], _} :> (Denominator[exp] - Numerator[exp])/
                    Numerator[exp], {Complex[
                     0, _], _} :> -1})] /@ (List @@ #)),
          #^(maxnestdepth[#,
               Function[{var}, MatchQ[var, Power[_, _Rational]]]][[
              1]] /. {{Power[base_,
                 exp_], _} :> (Denominator[exp] - Numerator[exp])/
                Numerator[exp], {Complex[0, _], _} :> -1})
          ],
         Times[a___, Alternatives[_Rational, _Integer], b___] :>
          a*b] & /@ Sort[terms, (Re[#1] <= Re[#2]) &], 1];
    multipliers = {{True, {0, Infinity, Infinity}}, multipliers};
    ];

   solution = problem;
   history = {{0, Infinity, Infinity}};
   (*multiplierstack form {{{<do full set>,{<multiplier,minpoly,
   degree for generating this set>}},{<multipliers>}},...}*)
   multiplierstack = {multipliers};
   (*add power set of multipliers as second element in multiplierstack*)
   (*multiplierstack=Append[multiplierstack,
   {multiplierstack[[1,1]],ComplexitySort[DeleteCases[Union[(Times@@#)&/@
   Subsets[DeleteCases[multipliers[[2]],1]]],mult_/;MemberQ[
   multipliers[[2]],mult]]]}];*)
   Catch[
    For[{i1 = 1}, i1 <= Length[multiplierstack], i1++,
     (*compute new multipliers from data if not already done*)
     If[ListQ[multiplierstack[[i1, 2]]] =!= True,
      multiplierstack[[i1]] =
        multiplierstack[[i1, -2]] @@ (multiplierstack[[i1, -1]]);
      ];
     multipliers = multiplierstack[[i1]];
     stackchange = False;
```

```
    If[multipliers[[1, 2]] =!= history[[1]],
     Print["multiplier count: ", Length[multipliers[[2]]], "  from: ",
       multipliers[[1, 2, {1, -1}]]],
     If[! ListQ[forcedmultipliers],
      Print["multiplier count: ", Length[multipliers[[2]]],
        "  multipliers: ", multipliers[[2]]];,
      Print["multiplier count: ", Length[multipliers[[2]]]];
      ]
     ];
    For[{i2 = 1}, i2 <= Length[multipliers[[2]]], i2++,
     multiplier = multipliers[[2, i2]];
     updatehistory = False;
     addmultipliers = False;
     (*try a multiplier, different based on history*)
     If[And[Length[history] >= 3, history[[-2]] =!= history[[1]]],
      attempt =
       trymultiplier[radicand, outerpower, multiplier,
        history[[-1, -1]], True],
      attempt =
        trymultiplier[radicand, outerpower, multiplier,
         history[[-1, -1]], False];
      ];
     multiplierstack[[i1, 2, i2]] = {multiplier, attempt[[2, -1]]};

     If[And[multipliers[[1, 1]] === True,
       attempt[[2, -1]] <= history[[-1, -1]]],
      addmultipliers = True
      ];
     (*check if progress has been made on this attempt,
     no action taken if no progress*)
     If[attempt[[1]] === True,
      (*history will be updated*)
      updatehistory = True;

      (*decide if a stackchange will be made*)
      If[history[[-1]] =!= history[[1]],
       stackchange = True;
       ];

      If[ListQ[attempt[[-1]]] =!= True,
       (*a solution was obtained*)
       Print["success: ",
        Join[attempt[[2, {1, -1}]],
         If[IntegerQ[multiplier], {FactorInteger[multiplier]}, {}]]];
       solution = Simplify[attempt[[-1]]];
       If[
        Or[history[[-1]] === history[[1]],
         history[[2, -1]]/attempt[[2, -1]] >= reductionpower],
        (*not worth trying for additional denesting, exit immediately*)
        history = Append[history, attempt[[2]]];
        Throw[, "solution"],
        (*further denesting will be attempted*)
        Print["attempting additional denesting, current solution is:"];
        Print[solution];
        ];
       ];

      If[
       And[history[[-1]] =!= history[[1]],
        Or[attempt[[2, -1]] < history[[-1, -1]],
         history[[-2]] === history[[1]]]],
       (*improvement has occured, new multipliers will be created*)
       Print["improvement  ", attempt[[2, {1, -1}]]];
       addmultipliers = True;
       ];
      ];

     (*add new multipliers if needed*)
     If[addmultipliers === True,
```

```
      Block[{smaller, larger, done, new},
       If[attempt[[2, -1]] <= history[[-1, -1]],
        smaller = attempt[[2]];
        larger = history[[-1]],
        smaller = history[[-1]];
        larger = attempt[[2]];
        ];
       new = {{False, smaller},
         newmultipliers, {smaller,
          If[larger =!= history[[1]], larger, Null], reductionpower}};

       Print["adding new multipliers from: ", multiplier];
       (*update multiplierstack with new multipliers*)
       If[multipliers[[1, 1]] === False,
        (*use new multipliers immediately*)
        done = {multipliers[[1]], multipliers[[2, 1 ;; i2]]};
        multiplierstack = Insert[multiplierstack, new, i1 + 1];
        If[i2 < Length[multipliers[[2]]],

         multiplierstack =
           Insert[multiplierstack, {multipliers[[1]],
             multipliers[[2, i2 + 1 ;; -1]]}, i1 + 2];
         ];
        multiplierstack[[i1]] = done,
        (*use new multipliers later*)
        stackchange = False;
        For[{i3 = Length[multiplierstack]}, i3 >= i1, i3--,

         If[Or[multiplierstack[[i3, 1, 2]] === history[[1]],
           new[[1, 2, -1]] > multiplierstack[[i3, 1, 2, -1]],
           And[new[[1, 2, -1]] === multiplierstack[[i3, 1, 2, -1]],
            LeafCount[new[[1, 2, 1]]] >=
             LeafCount[multiplierstack[[i3, 1, 2, 1]]]]],
          multiplierstack = Insert[multiplierstack, new, i3 + 1];
          Break[];
          ]
         ];
        ];
       ]
      ];

     (*update history if needed*)
     If[updatehistory === True,
      For[{i3 = Length[history]}, i3 >= 1, i3--,
        If[history[[i3, -1]] > attempt[[2, -1]],
         history = Insert[history, attempt[[2]], i3 + 1];
         Break[];
         ]
        ];
      ];

     (*break inner loop if stack has changed*)
     If[stackchange === True,
      Break[];
      ];
     ];(*end of inner loop*)

    (*add additional multipliers after first pass (if no forced multip\
liers)*)
    If[And[i1 === 1, ! ListQ[forcedmultipliers]],
     Block[{best, worst, extra, f},
      best = Min[multiplierstack[[i1, 2, All, 2]]];
      worst = Max[multiplierstack[[i1, 2, All, 2]]];
      extra =
       DeleteCases[
        DeleteDuplicates[
         Replace[(Distribute[
               f @@ (#^Range[0, 1] & /@ (Cases[
                    multiplierstack[[i1, 2]], {mult_, degree_} /;
```

```
                    degree < worst][[All, 1]])),
                List] //. {f[a___, Power[base_Times, exp_], b___] :>
                 f[a, b, Sequence @@ ((List @@ base)^exp)],
                f[a___, prod_Times, b___] :> f[a, b, Sequence @@ prod],
                 f[a___, Power[base_, exp_], b___] :>
                 f[a, b, (-1)^exp, (-base)^exp] /;
                  And[base != -1, Re[base] < 0],
                f[a___, Power[base_, Rational[num1_, denom_]], b___,
                  Power[base_, Rational[num2_, denom_]], c___] :>
                 f[a, b, c, base^(Mod[num1 + num2, denom]/denom)]}) /.
             f -> Times, Times[a___, _Integer, b___] :> a*b, {1}] //.
          Power[base_, Rational[a_, b_]] :> base^(Mod[a, b]/b)],
        Alternatives @@ multiplierstack[[i1, 2, All, 1]]];
      If[Length[extra] >= 1,
       Print["adding multipliers based on progress in initial guess"];
       multiplierstack =
        Insert[multiplierstack, {multiplierstack[[i1, 1]], extra},
         i1 + 1];
       ];
      ]
     ];

    ](*end of outer loop*),
   "solution"
   ];(*end of Catch, used to break the loop when a solution is found*)

  Print["history: ", #[[{1, -1}]] & /@ history];
  If[solution =!= problem,
   Print[solution],
   Print["no progress"];
   ];
  Print[];
  Return[solution];
  ]

ClearAll[RationalizeDenominator];
RationalizeDenominator[expr_] :=
 Block[{conjugate, denom, x},
  denom = Denominator[expr];
  conjugate = MinimalPolynomial[denom];
  Expand[
    Numerator[expr]*PolynomialQuotient[conjugate[x], x - denom, x] /.
     x -> 0]/(-1*conjugate[0])
  ]

ClearAll[AlgebraicQ];
AlgebraicQ[num_] := Element[num, Algebraics]

ClearAll[nestdepths]
nestdepths[expr_, func_] :=
 Block[{inner},
  If[Depth[expr] === 1,
   {expr, If[func[expr], 1, 0]},
   inner = Map[nestdepths[#, func] &, (List @@ expr), {1}];
   {expr,
    Sequence @@
     inner, ((Max @@ ((#[[-1]]) & /@ inner)) + If[func[expr], 1, 0])}
   ]
  ]

ClearAll[nestdepthsort]
nestdepthsort[expr_, func_] :=
 Block[{data},
  data = nestdepths[expr, func];
  data =
   data //.
    patt : {a : Except[_List], b : __, c : Except[_List]} :> {{a, c},
      b};
  data = data //. patt : {rep : Repeated[{_, _}]} :> Sequence @@ patt;
```

```
  data = Gather[{data}, (#1[[2]] == #2[[2]]) &];
  data = Sort[data, (#1[[1, 2]] >= #2[[1, 2]]) &];
  data
  ]

ClearAll[maxnestdepth]
maxnestdepth[expr_, func_] :=
 Block[{data, max},
  data = nestdepths[{expr}, func][[2]];
  max = data[[-1]];
  data =
   FixedPoint[
    DeleteCases[# /. {a_, b_} :> {} /; b < max, {}, Infinity] &, data];
  data =
   If[And[ListQ[#], Length[#] > 2], Sequence @@ (#[[2 ;; -2]]), #] & /@
     data;
  If[Length[data] > 2,
   data = data[[2 ;; -2]],
   data = {data}
   ];
  data
  ]

ClearAll[ComplexitySort];
ComplexitySort[expr_] :=
  Return[Sort[expr,
    Switch[LeafCount[#1] - LeafCount[#2], a_ /; a < 0, True,
      a_ /; a > 0, False, 0, Abs[#1] <= Abs[#2]] &]];

ClearAll[Factorc]
Factorc[expr_] :=
 Flatten[
   fullfactor[
     expr] //. {pow : {a : Except[_List], b : Except[_List]} :> {a^b},
      prod : {rep : (Repeated[{Except[_List]}, {2,
            Infinity}])} :> {Times[rep]},
     sum : {rep : Repeated[{{Except[_List]}}, {2, Infinity}]} :>
      Plus[rep],
     pow2 : {{a : Except[_List]}, b : Except[_List]} :> {a^b},
     pow3 : {{{a : Except[_List]}}, b : Except[_List]} :> {a^b}}][[1]]

ClearAll[fullfactor]
fullfactor[expr_] :=
 Block[{factorlist, i = 0, j = 0, done = False},
  factorlist = FactorList[expr];
  While[! done && j < 5,
   j++;
   done = True;
   For[{i = 1}, i <= Length[factorlist], i++,
    Which[
     Head[factorlist[[i, 1]]] === Power,
     done = False;
     factorlist[[
       i]] = {{factorlist[[i, 1, 1]],
        factorlist[[i, 1, 2]]*factorlist[[i, 2]]}},
     And[
      MemberQ[{Integer, Rational}, Head[factorlist[[i, 1]]]], !
       PrimeQ[factorlist[[i, 1]]], Abs[factorlist[[i, 1]]] =!= 1],
     done = False;
     factorlist[[i]] = {#[[1]], #[[2]]*factorlist[[i, 2]]} & /@
       FactorInteger[factorlist[[i, 1]]],
     Head[factorlist[[i, 1]]] === Plus,
     done = False;
     factorlist[[i]] =
      combine[fullfactor /@
        Replace[factorlist[[i, 1]], Plus[a_, b__] :> {a, b}],
       factorlist[[i, 2]]],
     True,
     factorlist[[i]] = {factorlist[[i]]}
```

```
      ]
     ];
    factorlist = Flatten[factorlist, 1]
    ];
   factorlist
   ]

ClearAll[combine];
combine[terms_List, pow_] :=
 Block[{simpler = terms, base, bases, positions, factor, i, j, k},
  For[{i = 1}, i <= Length[terms], i++,
   base = terms[[i]];
   For[{j = 1}, j < Length[base], j++,
    For[{k = j + 1}, k <= Length[base], k++,
     If[base[[j, 1]] === base[[k, 1]],
      base[[j]] = {base[[j, 1]], base[[j, 2]] + base[[k, 2]]};
      base[[k]] = {0, 0}
      ]
     ];
    If[base[[j, 1]] === 1,
     base[[j]] = {0, 0}
     ]
    ];
   simpler[[i]] = Delete[base, Position[base, {0, 0}]];
   ];
  bases = Intersection @@ Map[#[[1]] &, simpler, {2}];
  For[{i = 1}, i <= Length[bases], i++,
   positions =
    Position[#, {bases[[i]], Except[_List]}, 1] & /@ simpler;
   factor =
    Min[Extract[simpler[[#]], positions[[#]]][[1, 2]] & /@
      Range[1, Length[simpler], 1]];
   For[{j = 1}, j <= Length[simpler], j++,
    simpler[[j, Sequence @@ (positions[[j, 1]]), 2]] -= factor
    ];
   bases[[i]] = {bases[[i]], factor*pow};
   ];
  Append[bases, {simpler, pow}]
  ]
```

References

[1] Supplied source, *Pasted text(20260905-171712).txt*. Unmodified copy distributed as `source.wl`; 585 physical lines. Source-line references throughout this article identify the analyzed definitions directly.

[2] Wolfram Research, *Power*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Power.html. Accessed September 5, 2026.

[3] Wolfram Research, *MinimalPolynomial*. https://reference.wolfram.com/language/ref/MinimalPolynomial.html. Accessed September 5, 2026.

[4] Wolfram Research, *PolynomialGCD*. https://reference.wolfram.com/language/ref/PolynomialGCD.html. Accessed September 5, 2026.

[5] Wolfram Research, *PossibleZeroQ*. https://reference.wolfram.com/language/ref/PossibleZeroQ.html. Accessed September 5, 2026.

[6] Wolfram Research, *RootReduce*. https://reference.wolfram.com/language/ref/RootReduce.html. Accessed September 5, 2026.

[7] Wolfram Research, *Slot*. https://reference.wolfram.com/language/ref/Slot.html. Accessed September 5, 2026.

[8] Wolfram Research, *Sort*. https://reference.wolfram.com/language/ref/Sort.html. Accessed September 5, 2026.

[9] Wolfram Research, *FactorInteger*. https://reference.wolfram.com/language/ref/FactorInteger.html. Accessed September 5, 2026.

[10] Wolfram Research, *Block*. https://reference.wolfram.com/language/ref/Block.html. Accessed September 5, 2026.

[11] Wolfram Research, *Apply*. https://reference.wolfram.com/language/ref/Apply.html. Accessed September 5, 2026.

[12] Wolfram Research, *MemberQ*. https://reference.wolfram.com/language/ref/MemberQ.html. Accessed September 5, 2026.

[13] Wolfram Research, *ComplexExpand*. https://reference.wolfram.com/language/ref/ComplexExpand.html. Accessed September 5, 2026.

[14] Wolfram Research, *PowerExpand*. https://reference.wolfram.com/language/ref/PowerExpand.html. Accessed September 5, 2026.

[15] Wolfram Research, *Algebraics*. https://reference.wolfram.com/language/ref/Algebraics.html. Accessed September 5, 2026.